Universidad – Ingeniería Informática

Instalación y Mantenimiento de Computadores

Sistemas Empotrados II

---

# Sistema IoT para control y monitorización de un terrario

---

*[ Fotografía del montaje final del sistema ]*


**Autor:** Pablo Galilea Valverde


**Asignatura:** Instalación y Mantenimiento de Computadores

**Titulación:** Grado en Ingeniería Informática

**Fecha:** 28 de abril de 2026

**Entrega:** 15 de mayo de 2026

# Índice

**1. Introducción** **2**
1.1. Objetivos . . . . . 2
1.2. Alcance del documento . . . . . 2

**2. Descripción general del sistema** **3**
2.1. Arquitectura del sistema . . . . . 3
2.2. Sensores y actuadores . . . . . 3
2.3. Comunicaciones . . . . . 4

**3. Subsistema Arduino UNO** **5**
3.1. Descripción funcional . . . . . 5
3.2. Esquema eléctrico . . . . . 5
3.3. Descripción del circuito . . . . . 5
3.4. Diagrama de flujo del programa Arduino . . . . . 6
3.5. Código fuente Arduino . . . . . 6

**4. Subsistema Raspberry Pi 3B+** **7**
4.1. Descripción funcional . . . . . 7
4.2. Esquema eléctrico . . . . . 7
4.3. Descripción del circuito . . . . . 7
4.4. Diagrama de flujo del programa Raspberry Pi . . . . . 8
4.5. Código fuente Raspberry Pi . . . . . 8

**5. Interfaz en la nube: ThingSpeak** **9**
5.1. Descripción funcional . . . . . 9
5.2. Configuración del canal . . . . . 9
5.3. Diagrama de flujo de la comunicación con ThingSpeak . . . . . 9
5.4. Código fuente de la comunicación ThingSpeak . . . . . 10

**6. Presupuesto estimado** **11**

**7. Resultados y pruebas** **12**
7.1. Pruebas de validación . . . . . 12
7.2. Capturas de pantalla de ThingSpeak . . . . . 12
7.3. Fotografías del montaje . . . . . 12

**8. Conclusiones** **13**
8.1. Conclusiones técnicas . . . . . 13
8.2. Dificultades encontradas . . . . . 13
8.3. Trabajo futuro . . . . . 13

**9. Referencias** **14**

# 1. Introducción

El presente documento recoge la memoria técnica del proyecto final de la asignatura *Instalación y Mantenimiento de Computadores* (Sistemas Empotrados II), consistente en el diseño, implementación y documentación de un sistema IoT para la **automatización y monitorización de un terrario** de pequeñas dimensiones.

El sistema desarrollado permite vigilar y controlar las condiciones ambientales del terrario (temperatura, humedad y detección de fugas) garantizando el confort de los seres vivos que lo habitan. Para ello se integran dos plataformas de hardware embebido —un microcontrolador **Arduino UNO** y un minicomputador **Raspberry Pi 3B+**— junto con una plataforma de visualización en la nube mediante **ThingSpeak**.

## 1.1. Objetivos

- Diseñar e implementar un sistema de adquisición de datos con Arduino UNO que lea temperatura, humedad y distancia por ultrasonidos cada segundo mediante interrupción de *timer*.
- Controlar un ventilador y un sistema de iluminación mediante señal PWM, ajustables tanto manualmente (potenciómetros) como remotamente (ThingSpeak).
- Integrar una interfaz de usuario local con Raspberry Pi 3B+: pantalla LCD, botones de inicio/alarma e indicadores LED.
- Publicar y visualizar los datos del sistema en la plataforma ThingSpeak, con actualización cada 30 segundos.
- Detectar posibles fugas del animal mediante sensor de barrera de ultrasonidos en la puerta del terrario.

## 1.2. Alcance del documento

Esta memoria cubre los aspectos de diseño hardware (esquemas eléctricos), diseño software (diagramas de flujo y código fuente), presupuesto estimado del proyecto y las conclusiones obtenidas tras la realización del mismo.

# 2. Descripción general del sistema

## 2.1. Arquitectura del sistema

El sistema se compone de tres subsistemas principales que se comunican entre sí formando una arquitectura cliente-servidor en dos niveles:

1. **Subsistema Arduino UNO** (nodo sensor/actuador): adquiere datos de los sensores y controla los actuadores físicos del terrario.
2. **Subsistema Raspberry Pi 3B+** (nodo de control local): gestiona la comunicación serie con el Arduino, la interfaz de usuario local y la comunicación con la nube.
3. **Plataforma ThingSpeak** (interfaz remota en la nube): almacena y visualiza los datos, y permite el control remoto de actuadores.

*[ Diagrama de arquitectura general del sistema (bloques) ]*

## 2.2. Sensores y actuadores

La tabla 1 resume los sensores y actuadores del sistema con su función y el nodo al que están conectados.

Cuadro 1: Sensores y actuadores del sistema

| **Componente** | Tipo | Nodo | Función |
|---|---|---|---|
| **DHT11** | Sensor | Arduino | Temperatura y humedad relativa |
| **HC-SR04** | Sensor | Arduino | Distancia por ultrasonidos (detección de fuga) |
| **Potenciómetro 1** | Sensor | Arduino | Control manual de velocidad del ventilador |
| **Potenciómetro 2** | Sensor | Arduino | Control manual de intensidad de iluminación |
| **Ventilador (PWM)** | Actuador | Arduino | Regulación de temperatura |
| **Tira LED** | Actuador | Arduino | Iluminación del terrario |
| **Botón INICIO** | Entrada | RPi 3B+ | Arranque del sistema |
| **Botón ALARMA** | Entrada | RPi 3B+ | Activación/desactivación de alarma |
| **LED ESTADO** | Indicador | RPi 3B+ | Estado del sistema (activo/inactivo) |
| **LED ALARMA** | Indicador | RPi 3B+ | Estado de alarma |
| **Pantalla LCD** | Salida | RPi 3B+ | Visualización local de datos y estado |

### 2.3. Comunicaciones

- **Arduino ↔ Raspberry Pi**: comunicación serie UART (USB) a 9600 bps. El Arduino envía una trama de datos cada segundo con temperatura, humedad y distancia. La RPi envía comandos de control (velocidad de ventilador e intensidad de iluminación).
- **Raspberry Pi ↔ ThingSpeak**: protocolo HTTPS/REST sobre Wi-Fi. La RPi publica datos promediados cada 30 segundos y lee los comandos de usuario desde los campos de ThingSpeak.

# 3. Subsistema Arduino UNO

## 3.1. Descripción funcional

El Arduino UNO actúa como nodo sensor/actuador del sistema. Sus funciones principales son:

- Leer el sensor de temperatura y humedad **DHT11** y el sensor de ultrasonidos **HC-SR04 una vez por segundo** mediante la interrupción generada por un *timer* hardware.
- Leer los dos **potenciómetros** analógicos para el control manual de velocidad de ventilador e intensidad de iluminación.
- Generar señales **PWM** para controlar el ventilador y la iluminación a través de transistores MOSFET IRL540n.
- Enviar los datos adquiridos a la Raspberry Pi por **puerto serie** y recibir los valores de control remoto.
- Gestionar la lógica de **alarma**: si el sensor de ultrasonidos detecta un objeto a menos de 10 cm, se paran todos los actuadores.

## 3.2. Esquema eléctrico

*[ Esquema eléctrico del subsistema Arduino UNO (DHT11, HC-SR04, potenciómetros, ventilador PWM con MOSFET IRL540n, tira LED con MOSFET IRL540n) ]*

*[ Detalle: circuito de control PWM con transistor MOSFET IRL540n (resistencias R1, R2, R3 y componentes asociados) ]*

Figura 1: Control PWM mediante transistor MOSFET IRL540n

## 3.3. Descripción del circuito

El control de los actuadores (ventilador y LED) se realiza mediante transistores MOSFET de canal N modelo **IRL540n**. Cada circuito de control incluye:

- **R1**: resistencia de puerta ($\approx 220\,\Omega$) para limitar la corriente transitoria y proteger el pin GPIO del Arduino.

- **R2**: resistencia de *pull-down* ($\approx 10\,\mathrm{k}\Omega$) para mantener la puerta a nivel bajo cuando el pin está en alta impedancia.
- **R3**: resistencia de limitación de corriente del LED de estado.
- La señal PWM generada por el Arduino varía el ciclo de trabajo del MOSFET, regulando la potencia entregada al actuador.

### 3.4. Diagrama de flujo del programa Arduino

*[ Diagrama de flujo / UML del programa Arduino (setup, loop principal, ISR del timer, rutina de lectura DHT11, rutina HC-SR04, envío serie, recepción de comandos, gestión de alarma) ]*

### 3.5. Código fuente Arduino

*[ Código fuente completo del programa Arduino (.ino) (inclusión de librerías, definición de pines, setup(), loop(), ISR Timer1, funciones auxiliares) ]*

# 4. Subsistema Raspberry Pi 3B+

## 4.1. Descripción funcional

La Raspberry Pi 3B+ actúa como nodo de control central del sistema. Sus funciones son:

- Leer los datos enviados por el Arduino a través del **puerto serie** cada segundo (temperatura, humedad, distancia).
- Mostrar en la **pantalla LCD** la temperatura, humedad, estado del sistema y estado de la alarma.
- Gestionar los dos **botones físicos**: botón de inicio del sistema y botón de activación/desactivación de alarma.
- Controlar los dos **LEDs indicadores** de estado del sistema y alarma.
- Enviar al Arduino los valores de control (velocidad del ventilador e intensidad de la iluminación) recibidos desde ThingSpeak.
- Publicar en **ThingSpeak** los datos promediados cada 30 segundos.
- Activar la **alarma** automáticamente si la distancia medida por el sensor de ultrasonidos es inferior a 10 cm.

## 4.2. Esquema eléctrico

*[ Esquema eléctrico del subsistema Raspberry Pi 3B+ (conexión GPIO: botones con pull-up/pull-down, LEDs con resistencias limitadoras, pantalla LCD I2C/paralelo, conexión serie al Arduino) ]*

## 4.3. Descripción del circuito

La Raspberry Pi 3B+ opera a **3,3 V** en sus pines GPIO, por lo que se deben respetar los niveles lógicos en todas las conexiones:

- **Botones**: conectados con resistencias de *pull-up* internas del SoC BCM2837. La pulsación lleva el pin a nivel bajo (lógica inversa).
- **LEDs indicadores**: conectados con resistencia limitadora de corriente ($\approx 330\,\Omega$) entre el pin GPIO y el LED.
- **Pantalla LCD**: interfaz $I^2C$ (SDA/SCL, pines GPIO 2 y 3).
- **Comunicación serie**: conexión USB al Arduino UNO (`/dev/ttyACM0`).

### 4.4. Diagrama de flujo del programa Raspberry Pi

*[ Diagrama de flujo / UML del programa Raspberry Pi (hilo principal, hilo lectura serie, hilo actualización LCD, hilo ThingSpeak, gestión de interrupciones GPIO, máquina de estados del sistema) ]*

### 4.5. Código fuente Raspberry Pi

*[ Código fuente completo del programa Raspberry Pi (Python) (importaciones, configuración GPIO, configuración serie, bucle de lectura, hilo ThingSpeak, gestión de botones, actualización LCD) ]*

# 5. Interfaz en la nube: ThingSpeak

## 5.1. Descripción funcional

ThingSpeak es la plataforma IoT utilizada para la visualización remota de datos y el control de los actuadores del terrario. Se configura un *channel* con los siguientes campos (*fields*):

Cuadro 2: Configuración de campos en ThingSpeak

| Field | Nombre | Descripción |
|---|---|---|
| Field 1 | Temperatura (°C) | Valor promedio en 30 s |
| Field 2 | Humedad (%) | Valor promedio en 30 s |
| Field 3 | Distancia ultrasonidos (cm) | Valor promedio en 30 s |
| Field 4 | Estado alarma | 0 = normal, 1 = alarma |
| Field 5 | Velocidad ventilador (%) | Comando de usuario (0–100) |
| Field 6 | Intensidad iluminación (%) | Comando de usuario (0–100) |
| Field 7 | Estado ventilador | 0 = apagado, 1 = encendido |
| Field 8 | Estado iluminación | 0 = apagada, 1 = encendida |

## 5.2. Configuración del canal

- **Periodicidad de escritura**: cada 30 segundos (promedio de los datos recogidos durante ese intervalo).
- **Periodicidad de lectura**: cada 30 segundos para obtener los comandos de control del usuario.
- **Visualización**: gráficas de temperatura, humedad y distancia con indicadores de estado de ventilador, iluminación y alarma.
- **Control**: widgets de botón y selector numérico para activar el ventilador e iluminación y ajustar su nivel.

## 5.3. Diagrama de flujo de la comunicación con ThingSpeak

*[ Diagrama de flujo / UML de la comunicación con ThingSpeak (autenticación API key, promediado de datos, escritura HTTP REST, lectura de comandos, lógica de actualización) ]*

### 5.4. Código fuente de la comunicación ThingSpeak

*[ Fragmento de código Python para lectura/escritura en ThingSpeak (función write_thingspeak(), función read_thingspeak(), gestión de errores HTTP) ]*

## 6. Presupuesto estimado

La tabla 3 recoge el coste estimado de los componentes hardware necesarios para la realización del proyecto.

Cuadro 3: Presupuesto estimado de componentes hardware

| **Componente** | **Ud.** | **Ref.** | **P. unit.** | **Total** |
|---|---|---|---|---|
| Arduino UNO R3 | 1 | – | 22,00 € | 22,00 € |
| Raspberry Pi 3B+ | 1 | – | 45,00 € | 45,00 € |
| Sensor DHT11 | 1 | DHT11 | 2,50 € | 2,50 € |
| Sensor ultrasonidos HC-SR04 | 1 | HC-SR04 | 2,00 € | 2,00 € |
| Transistor MOSFET IRL540n | 2 | IRL540n | 0,80 € | 1,60 € |
| Potenciómetro 10 kΩ | 2 | – | 0,40 € | 0,80 € |
| Ventilador 5 V | 1 | – | 3,50 € | 3,50 € |
| Tira LED | 1 | – | 4,00 € | 4,00 € |
| Pantalla LCD 16×2 I2C | 1 | – | 5,00 € | 5,00 € |
| Pulsadores | 2 | – | 0,30 € | 0,60 € |
| LEDs indicadores | 2 | – | 0,10 € | 0,20 € |
| Resistencias (surtido) | 1 | – | 1,50 € | 1,50 € |
| Cables y protoboard | 1 | – | 5,00 € | 5,00 € |
| Tarjeta microSD (16 GB) | 1 | – | 6,00 € | 6,00 € |
| | | | **TOTAL estimado** | **100,20 €** |

**Nota**: los precios son orientativos y pueden variar según el proveedor. No se incluyen costes de mano de obra ni herramientas (soldador, multímetro, etc.).

# 7. Resultados y pruebas

## 7.1. Pruebas de validación

Para verificar el correcto funcionamiento del sistema se han realizado las siguientes pruebas:

1. **Lectura de sensores**: se comprueba que el Arduino lee el DHT11 y el HC-SR04 cada segundo mediante interrupción del timer y envía los datos correctamente por el puerto serie.
2. **Control PWM**: se verifica que la señal PWM enviada al ventilador y al LED varía de forma proporcional al valor del potenciómetro o al comando recibido desde ThingSpeak.
3. **Detección de alarma**: se coloca un objeto a menos de 10 cm del sensor HC-SR04 y se comprueba que el sistema activa la alarma, enciende el LED de alarma, detiene los actuadores y muestra el estado en la pantalla LCD.
4. **Comunicación serie**: se valida la correcta recepción de tramas de datos por la Raspberry Pi y el reenvío de comandos al Arduino.
5. **Publicación en ThingSpeak**: se comprueba que los datos aparecen en el canal de ThingSpeak cada 30 segundos y que los comandos de usuario se aplican correctamente.
6. **Interfaz local**: se verifica el correcto funcionamiento de la pantalla LCD, los botones y los LEDs indicadores.

## 7.2. Capturas de pantalla de ThingSpeak

*[ Capturas de pantalla del canal ThingSpeak (gráficas de temperatura, humedad, distancia; widgets de control de ventilador e iluminación) ]*

## 7.3. Fotografías del montaje

*[ Fotografías del montaje hardware (protoboard con Arduino, conexiones RPi, terrario completo con sensores y actuadores instalados) ]*

# 8. Conclusiones

## 8.1. Conclusiones técnicas

Tras el desarrollo del proyecto se extraen las siguientes conclusiones técnicas:

- El uso de la **interrupción de timer** en Arduino garantiza la periodicidad exacta de 1 segundo en la lectura de sensores, independientemente de la carga del bucle principal, lo que es fundamental para la fiabilidad del sistema de monitorización.
- El control **PWM mediante MOSFET IRL540n** ofrece una solución eficiente y de bajo coste para la regulación de potencia en actuadores de corriente continua, con una respuesta lineal y baja disipación de calor en el transistor.
- La arquitectura **Arduino + Raspberry Pi** permite separar la lógica de control en tiempo real (Arduino) de la lógica de comunicación y gestión de alto nivel (RPi), simplificando el diseño de cada nodo y mejorando la modularidad del sistema.
- La plataforma **ThingSpeak** facilita enormemente la integración IoT, proporcionando almacenamiento, visualización y API REST sin necesidad de infraestructura propia.

## 8.2. Dificultades encontradas

- La sincronización entre la interrupción del timer, la lectura del DHT11 (que requiere un protocolo de comunicación con tiempos precisos) y el envío serie fue uno de los aspectos más delicados del diseño del firmware de Arduino.
- La limitación de la API de ThingSpeak (mínimo 15 segundos entre escrituras en la versión gratuita) condicionó el diseño del protocolo de actualización.

## 8.3. Trabajo futuro

Como posibles mejoras del sistema se proponen:

- Añadir un sensor de luminosidad para ajuste automático de la iluminación.
- Implementar control PID para la regulación de temperatura.
- Migrar la comunicación Arduino–RPi a un protocolo más robusto (JSON sobre UART con checksum).
- Añadir notificaciones *push* ante eventos de alarma.

## 9. Referencias


1. Arduino. *Arduino UNO Reference*. https://docs.arduino.cc/hardware/uno-rev3/
2. Raspberry Pi Foundation. *Raspberry Pi 3B+ documentation*. https://www.raspberrypi.com/documentation/
3. Aosong Electronics. *DHT11 Humidity & Temperature Sensor Datasheet*.
4. SparkFun. *HC-SR04 Ultrasonic Distance Sensor Datasheet*.
5. International Rectifier. *IRL540n HEXFET Power MOSFET Datasheet*.
6. MathWorks. *ThingSpeak Documentation*. https://www.mathworks.com/help/thingspeak/
7. RPi.GPIO. *RPi.GPIO Python Library Documentation*. https://sourceforge.net/projects/raspberry-gpio-python/
8. Adafruit. *DHT11 Python Library*. https://github.com/adafruit/Adafruit_Python_DHT